
POTENS

IT Services and Consultancy Pvt. Ltd.

INTERNSHIP HIRING · 2026

The 24-Hour Take-Home.

For Frontend, Backend, and AI/ML candidates. Pick one. Ship it. Send it back.

Three roles. Two questions each. One choice.

CONFIDENTIAL · DO NOT SHARE OR POST PUBLICLY · POTENS IT SERVICES AND CONSULTANCY PVT. LTD. · PUNE

Hi. I'm Ninad.

I started Potens because there are categories of software where the standard of excellence has been low for too long, where the people using the product carry a lot of responsibility, and where the cost of a careless build shows up in someone else's day. The work is high-leverage. The trust we earn from customers is hard to come by, and easy to lose.

This internship is not coffee runs. If you join, you will write code that goes into a product I personally review. You will see your work used by real people inside a quarter. The bar is high because the stakes are real, and so is the trust we extend on day one.

"We don't grade on polish. We grade on taste, judgment, speed, and whether you're honest about what you used and what you didn't."

Below is a 24-hour take-home. Pick one question in your role. Build it like you mean it. Tell us how you thought, not just what you shipped.

If you read this far and the vibe is off, no hard feelings, skip it. If it is on, see you on the other side.

How this works.

Eight short rules. Read once. Follow properly.

1. 24 hours.

The clock starts when you open the assignment email. The assignment is not hard. It is designed for one focused day. Don't spend three.

2. Pick exactly one question.

You get two questions in your role. Pick the one that excites you more. Do not attempt both. We will not score the second.

3. Ship it on GitHub.

Repo name: **potens-intern-[role]-[your-name]** (example: *potens-intern-backend-rohan-mehta*).
Make it public, or share with **@ninadsumant** as a collaborator.

4. Commit like you're thinking out loud.

Incremental commits with clear messages. No single 3 a.m. dump. We will read your history. That is where we see how you actually work.

5. Document like a grown-up.

README must include how to run it, the design decisions you made, what is broken or unfinished, and what you would build next. Comment the non-obvious parts of your code. Skip the obvious ones. We know what a for-loop does.

6. AI tools, yes. But log them.

AI USE LOG required at the bottom of your README. We expect you to use AI assistants. That is how good engineers work in 2026. We just want honesty. List every tool you used (Claude, Cursor, Copilot, Bolt, v0, ChatGPT, Codex, anything else), an approximate token or message count for each, and one line on what you used it for. Lying about it is the fastest track to a no.

7. Submit via the Google Form.

Submission link: <https://forms.gle/5J4B1G8K8NWkBP4N9> The form will ask for your GitHub link, a 150-word approach summary, and your AI use log. Do not email submissions. They will be missed.

8. Questions? WhatsApp, not email.

WhatsApp: +91 95526 27897 · Click-to-chat: <https://wa.me/919552627897> Stuck on setup, the brief is unclear, the link is broken, anything at all. Message, do not spiral. Replies usually within an hour during the day.

Frontend.

What we're testing: taste in interfaces, command of React, intuition for multilingual and low-bandwidth users, and the ability to make a serious product feel *good* to use.

Q1. Multilingual Civic-Style PWA

Build a 3-screen mobile-first PWA where a user can pick a category, describe an issue (text, photo, optional voice), and receive a confirmation with a reference ID. Bilingual UI in English plus one Indian language of your choice. Slow-3G tolerant. Installable. The point is taste, not features.

REQUIRED

- Three screens flowing in order: category, details, confirmation.
- Full bilingual toggle. Every label, button, and placeholder must switch.
- Voice input on the details screen using the Web Speech API.
- Works without falling apart on Chrome DevTools throttled Slow 3G.
- PWA: manifest, service worker, installable to home screen. Submissions persist in localStorage. No backend required.
- One thoughtful micro-interaction. Should feel considered, not playful.
- Tailwind preferred. Vanilla CSS acceptable if you have strong opinions about it.

STRETCH (OPTIONAL)

- Offline-first: form usable with no network, submissions queued and synced when back online.
- WCAG 2.2 AA pass on Lighthouse, with the report screenshot in the README.
- A status tracker view that visualises four states with a clean timeline UI.

Q2. Operations Cockpit Dashboard

A senior operations user opens this dashboard at 9 a.m. before their day starts. Build that dashboard. The brief is intentionally broad. Make smart design calls and defend them in your README.

REQUIRED

- A top 5 list of items that need action today, with one-line context per item and approve / hold actions.
- A panel of anomalies flagged by an automated system. Mock JSON data is fine.
- A live counter for a time-sensitive metric of your choice (must be ticking).
- Full bilingual toggle: English and one Indian language of your choice.
- Desktop-optimised, responsive on tablet. Clean, considered. No clutter, no gimmicks.
- Tailwind required.

STRETCH (OPTIONAL)

- Keyboard shortcuts for power users (j and k navigation, a to approve, h to hold).
- A low-bandwidth toggle that drops images and switches to a sparser layout.
- A dark mode that does not feel cosmetic.

→ Pick ONE, Q1 or Q2, not both. We will not grade the second one.

Backend.

What we're testing: system thinking, API design, data modelling, and whether your code holds up when it is reviewed for *audit*, not for aesthetics.

Q1. Tamper-Evident Append-Only Log Service

Build a service where every event is written once and can never be edited or deleted. The chain must be self-verifiable so a third party can prove the log has not been tampered with. Node and Express with PostgreSQL preferred. SQLite acceptable if you justify it in the README.

REQUIRED

- POST /log accepts { actor, action, payload }. Each entry stores a SHA-256 chain hash that links to the previous entry. No edits, no deletes.
- GET /log/:id returns the entry plus a chain-verification status.
- GET /verify scans the entire chain and returns pass or fail, with the first broken entry id if any.
- GET /export returns a filtered JSON export (by date range and actor).
- Structured logging using pino or winston. A rate limit on POST. Database migrations checked in.
- A small auth layer (API key in a header is fine).

STRETCH (OPTIONAL)

- Merkle-tree style batching so /verify stays fast on 100k+ entries.
- A CLI command (npm run verify) that runs the full chain check.
- A docker-compose so we can boot the whole thing with one command.

Q2. Profile-to-Recommendation API

A user sends a structured profile and you return the top three items from a catalogue that match them, with reasoning. Pick the domain yourself. Build a non-trivial catalogue of at least 15 items. We are watching how you model eligibility rules and how clearly you explain a match.

REQUIRED

- POST /recommend takes a profile (you decide the fields, minimum five attributes) and returns a ranked top three with a one-paragraph reason for each match.
- GET /items (admin token protected) returns full CRUD.
- GET /explain/:item_id returns the eligibility logic in plain language.
- At least five unit tests covering boundary, missing-field, and no-match cases.
- A seed script that loads the 15+ item catalogue into the database.

STRETCH (OPTIONAL)

- A /subscribe webhook so a user is notified when newly added items match their profile.
- A caching layer with a sensible TTL on the recommend endpoint.
- OpenAPI spec auto-generated from the routes.

→ Pick ONE, Q1 or Q2, not both. We will not grade the second one.

AI / ML.

What we're testing: how you wire LLM components end-to-end, your honesty about model limits, and whether your prompts and traces look like a person who thought about it rather than someone who copied a tutorial.

Q1. Document Q&A with Citations

Build a RAG system over five or more documents of your choice. Any domain. Anything substantive (policy text, product docs, research papers, technical guides). The point is your retrieval, your chunking strategy, and your discipline about not letting the model hallucinate when the docs don't cover the answer.

REQUIRED

- Ingest, chunk, embed, and store the documents. Explain your chunking strategy in the README.
- An /ask endpoint that returns answers with citations. Each citation must include the source file, the chunk or page reference, and the snippet used.
- A /contradict endpoint that takes two document IDs and returns whether they conflict on a topic, with reasoning.
- A multilingual flow: a query in one language returns an answer in the same language. A translation step at the boundary is acceptable for the 24-hour version.
- A simple Streamlit or Gradio UI so we can try it without Postman.
- No silent hallucination. If the docs do not cover the question, the system must say so explicitly.
- Any vector store (Chroma, FAISS, pgvector). Any LLM with a free tier (Groq, Gemini, OpenAI free credits).

STRETCH (OPTIONAL)

- A confidence score per answer with a human-in-the-loop gate when confidence falls below a threshold.
- A reranker layered on top of vector retrieval.
- An eval set of 10 Q&A pairs with ground truth, scored on retrieval at top-k.

Q2. Triage Agent with Real Tool Calling

Build an agentic system that takes a free-text input (the input could be a complaint, a request, a ticket, you decide), and produces a structured triage decision. Use real tool calling, not a string-matching shortcut.

REQUIRED

- Input: free text plus optional metadata.
 - Output: { category, priority, next_tool, reasoning }. You define four to six categories and a P0 / P1 / P2 priority scheme.
 - Three callable tools that the agent picks from. You implement them as real functions. Examples: a lookup tool, an acknowledgment-drafting tool, a similar-past-input search tool. Pick what fits your problem framing.
 - The full reasoning trace must be visible for every decision. Not just the final answer.
-

- An /examples folder with at least ten test inputs and the agent's outputs.
- A "why" explanation field on every output. No silent magic.

STRETCH (OPTIONAL)

- A low-confidence escalation path that calls a human-in-the-loop tool when the agent is unsure.
- Run the same ten examples through a baseline (single-prompt classifier) and report the numbers side by side.
- A small Streamlit UI that visualises the reasoning trace as a tree.

→ Pick ONE, Q1 or Q2, not both. We will not grade the second one.

How we evaluate.

No rubric is perfect, but this is what we actually weigh when we sit down with your submission.

Taste & judgment	35%	Did you make smart choices when the brief had gaps? Does the thing feel considered, or generated?
Working code	25%	Does it run from a clean clone in under ten minutes following your README?
Honesty	15%	Clear about what works, what doesn't, and what was AI-assisted. We respect transparency more than perfection.
Speed & focus	15%	You picked one thing and shipped it well, not three things half-built.
Communication	10%	README, commit history, and your Loom walkthrough tell a coherent story.

A FEW HONEST THINGS

Before you start.

You are not competing on lines of code. A focused, working, well-documented submission beats a sprawling unfinished one every time. We mean that.

If you finish in eight hours, that is a flex, not a problem. Use the rest of the time on your README, your Loom, and your AI use log. Polish the parts we read.

If you do not finish a stretch goal, say so in the README. Tell us what you would have done. That is 80% of the value of attempting it.

If you used an AI assistant heavily, say so. We use them ourselves. The interns we have regretted hiring are the ones who hid it, not the ones who used it well.

And the obvious one: please do not copy from someone else's public repo. We run a quick check. It is genuinely the only thing that gets people instantly disqualified.

"If you're still reading and the idea of building something that ships into a real customer environment in 90 days feels exciting, we should talk. Ship something, then ping me."

See you on the other side,

Ninad Sumant

Co-founder & CEO · Potens IT Services and Consultancy Pvt. Ltd.

WhatsApp for queries: **+91 95526 27897**